

AVANTAJE ȘI DEZAVANTAJE ALE ARHITECTURILOR CQRS ȘI EVENT SOURCING COMPARATIV CU MODELUL CRUD

Ion BORTĂ¹

¹Departamentul Ingineria Software și Automatică, grupa TI-241M, Facultatea Calculatoare, Informatică și Microelectronică, Universitatea Tehnică a Moldovei, Chișinău, Republica Moldova

Autorul corespondent: Ion Bortă, ion.borta@isa.utm.md

Rezumat. Această lucrare analizează comparativ arhitectura tradițională de tip CRUD și cele moderne bazate pe CQRS (Command Query Responsibility Segregation) și Event Sourcing, cu scopul de a identifica avantajele, dezavantajele și scenariile potrivite de utilizare pentru fiecare abordare. Studiul a fost realizat pe baza unei evaluări teoretice, folosind criterii precum complexitatea sistemului, scalabilitatea, consistența datelor, performanța, posibilitatea de audit și efortul de dezvoltare. Rezultatele evidențiază că arhitectura CRUD este potrivită pentru aplicații simple și medii, oferind implementare rapidă și actualizare imediată a datelor, dar cu posibilități reduse de ajustare și analiză istorică a datelor. În schimb, CQRS combinat cu Event Sourcing permite o scalare eficientă a operațiilor de citire și scriere, păstrează toate modificările efectuate asupra stării sistemului și facilitează reconstruirea acestuia la orice moment în timp. Totuși, aceste beneficii vin cu un cost mai mare de complexitate și întreținere. Lucrarea scoate în evidență diferențele cheie dintre cele două abordări, organizate într-un tabel comparativ. Concluzia principală este că alegerea arhitecturii trebuie să fie ghidată de nevoile reale ale sistemului, nu de tendințe tehnologice.

Cuvinte cheie: scalabilitate, consistența datelor, audit, complexitatea sistemului, performanță

Introducere

Evoluția dezvoltării de software a fost mereu determinată de nevoia de a crea sisteme mai scalabile, mai ușor de întreținut și mai receptivă la cerințele de afaceri în schimbare. Unul dintre aspectele fundamentale care influențează aceste calități este alegerea arhitecturii sistemului. Una dintre cele mai durabile abordări arhitecturale este modelul CRUD (Create, Read, Update, Delete), care oferă un mecanism simplu pentru interacțiunea cu o bază de date. Implementarea sa simplă a contribuit la adoptarea sa pe scară largă într-o varietate de sisteme, în special în cele cu reguli de afaceri relativ simple.

Cu toate acestea, pe măsură ce sistemele cresc în complexitate și nevoia de scalabilitate, receptivitate și toleranță la erori crește, arhitecturile CRUD tradiționale prezintă deseori limitări obișnuite. Gestionarea accesului concurrent, asigurarea consistenței datelor în medii distribuite și adaptarea la modelele de afaceri în evoluție pot deveni semnificativ dificile în cadrul unei abordări bazate exclusiv pe CRUD.

CQRS introduce o separare clară între operațiunile care modifică datele (comenzi) și cele care recuperează datele (interogări), permițând modele optimizate pentru fiecare problemă. Event Sourcing, pe de altă parte, mută accentul de la stocarea stării curente a unei entități la salvarea unei secvențe cronologice de evenimente care descriu toate modificările aduse stării sistemului.

Acest articol are ca scop investigarea și compararea principalelor paradigme arhitecturale. Prin analiza punctelor forte și a limitărilor fiecărei abordări, se urmărește oferirea unei perspective cuprinzătoare asupra momentelor și motivelor pentru care dezvoltatorii pot opta pentru CQRS și Event Sourcing în favoarea arhitecturii CRUD tradiționale sau invers.

Noțiuni teoretice

CRUD, prescurtat de la Create (Creare), Read (Citire), Update (Actualizare) și Delete (Ștergere), este un model de proiectare fundamental care mapează operațiile de bază ale unei aplicații la baza de date utilizată [1]. Fiecare entitate din sistem corespunde, de obicei, unui tabel, iar fiecare operație modifică în mod direct starea curentă a datelor. Această abordare oferă simplitate și un ciclu rapid de dezvoltare, fiind potrivită pentru sisteme cu logică de afaceri simplă și cerințe reduse de concurență. O reprezentare vizuală a acestui model este prezentată în Fig. 1.

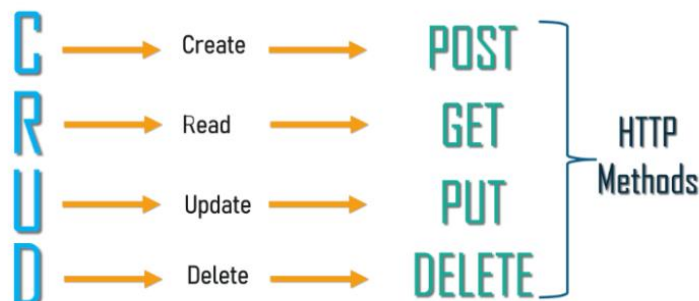


Figura 1. Operațiile CRUD

Command Query Responsibility Segregation (CQRS) este un model arhitectural care separă operațiile responsabile de modificarea datelor (**comenzi**) de cele responsabile de citirea acestora (**interogări**) (vezi Fig. 2). Această separare permite scalarea independentă a modelelor de citire și de scriere, optimizarea accesului la date în funcție de tipul operației și o delimitare mai clară a comportamentelor sistemului [2].

Prin decuplarea citirilor de scrieri, CQRS permite sistemelor să gestioneze o logică de domeniu mai complexă și să adopte mecanisme diferite de stocare pentru comenzi și interogări. Cu toate acestea, modelul introduce o complexitate suplimentară, necesitând deseori sincronizare atentă între modele și mecanisme sofisticate de gestionare a evenimentelor, în special în medii distribuite.

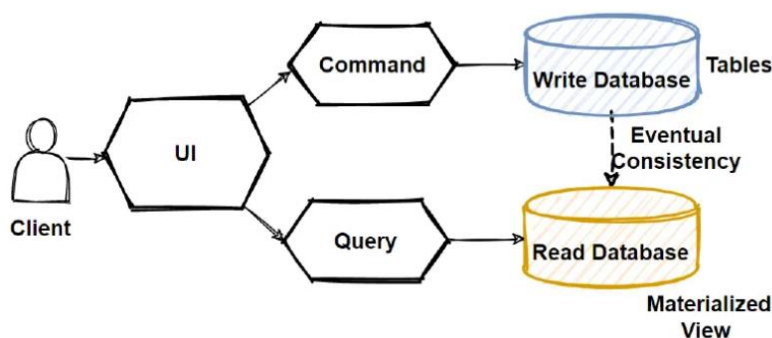


Figura 2. Modelul CQRS

Event Sourcing, prescurtat ES, este un model de proiectare în care schimbările aduse stării aplicației sunt persistate sub forma unei secvențe de evenimente, în loc de suprascrierea stării curente (vezi Fig. 3). Fiecare eveniment reprezintă un fapt care a avut loc, iar starea curentă a sistemului poate fi reconstruită prin reluarea (replaying) acestor evenimente [3].

Event Sourcing este integrat cu CQRS, în special în scenarii în care auditul istoric, interogările temporale și reluarea stării sunt importante. Acesta facilitează o istorie completă a modificărilor, oferind perspective valoroase asupra comportamentului sistemului în timp. Totuși,

Event Sourcing implică propriile provocări, inclusiv versionarea evenimentelor, cerințe sporite de stocare și necesitatea gestionării consistenței eventuale.

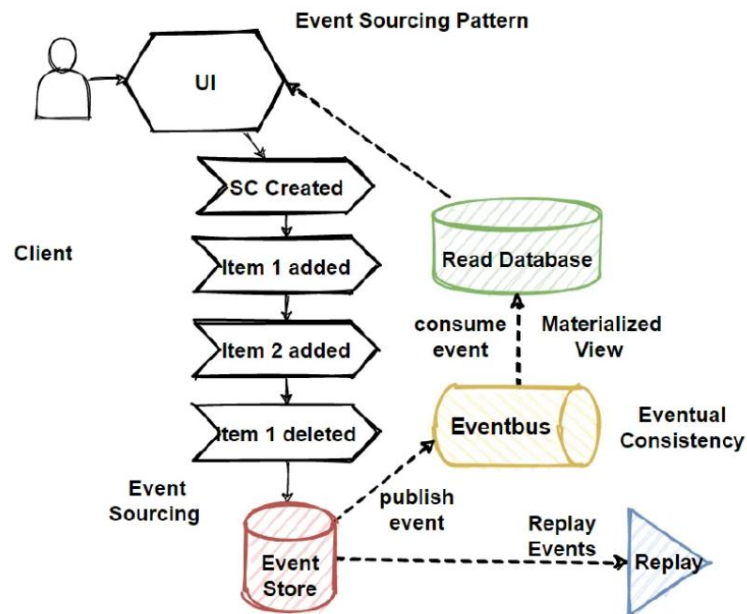


Figura 3. Modelul Event Sourcing

Analiza comparativă pe criterii

1. Complexitatea sistemului

CRUD este simplu, iar maparea directă dintre operațiuni și acțiunile bazei de date îl face ușor de înțeles, implementat și întreținut, în special pentru aplicațiile mici și mijlocii. Modelul permite implementarea rapidă a cerințelor de afaceri, cu un efort redus de proiectare arhitecturală.

CQRS și Event Sourcing introduc o complexitate semnificativă. Dezvoltatorii trebuie să gestioneze modele separate pentru comenzi și interogări, să gestioneze procesele asincrone și să facă față provocărilor precum consistența eventuală, ordonarea mesajelor și versionarea evenimentelor. Mai mult, înțelegerea și depanarea unor astfel de sisteme necesită cunoștințe arhitecturale mai aprofundate [4].

2. Scalabilitatea

Scalarea aplicațiilor CRUD poate fi dificilă, mai ales atunci când atât citirile, cât și scrierile concurează pentru aceleași resurse ale bazei de date. Bazele de date monolitice tradiționale devin adesea blocaje, necesitând soluții complexe, cum ar fi fragmentarea sau replicarea bazelor de date.

CQRS susține în mod natural scalabilitatea permițând scalarea independentă a laturilor de citire și scriere. Event Sourcing îmbunătățește și mai mult scalabilitatea prin activarea arhitecturilor bazate pe evenimente și a proceselor de decuplare. Sistemele pot scala cu ușurință modelele de citire pe orizontală pentru a acomoda interogări cu volum mare fără a afecta performanța de scriere [5,6,7].

3. Consistența și integritatea datelor

Sistemele CRUD se bazează de obicei pe garanții tranzacționale puternice (proprietăți ACID) furnizate de bazele de date relaționale. Consistența datelor este imediată și impusă de baza de date, ceea ce simplifică raționamentul despre starea sistemului.

CQRS adoptă adesea consecvența eventuală, în special în configurațiile distribuite. Comenzile se pot finaliza înainte ca rezultatele lor să fie reflectate în modelul de interogare. Event Sourcing înregistrează evenimente imutabile, asigurându-se că nu se pierd date, dar menținerea modelelor de citire actualizate introduce probleme suplimentare de sincronizare [5,6].

4. Performanța

Sistemele CRUD funcționează adecvat sub sarcini moderate. Cu toate acestea, pe măsură ce cererea crește, concurența în baza de date poate cauza degradarea performanței, în special pentru sistemele cu citire intensă.

Prin separarea responsabilităților de citire și scriere, CQRS îmbunătățește performanța prin optimizarea independentă a fiecărei părți. Modelele de citire pot fi reglate special pentru interogări rapide, în timp ce modelele de scriere se concentrează pe validarea și procesarea domeniului. Event Sourcing permite adăugarea eficientă a evenimentelor, mai degrabă decât actualizări complexe de stare [6,7].

5. Auditul și analiza istorică

Sistemele CRUD suprasciu de obicei datele în timpul actualizărilor, pierzând contextul istoric, cu excepția cazului în care sunt concepute explicit pentru a arhiva modificările (de exemplu, utilizând tabele de audit sau declanșatoare). Implementarea unei analize istorice complete necesită eforturi și instrumente suplimentare.

În Event Sourcing, întreaga istorie a tuturor modificărilor este păstrată sub forma unei secvențe de evenimente. Acest lucru oferă un istoric de audit natural și permite reconstruirea stării sistemului în orice moment din trecut, susținând astfel scenarii precum analiza forensică, revenirea la o stare anterioară sau conformitatea cu cerințele de audit. Analiza forensică se referă la procesul de colectare, examinare și interpretare a datelor digitale pentru a înțelege ce s-a întâmplat într-un sistem informatic, în special în cazul unor incidente de securitate, erori critice sau comportamente neașteptate [7].

6. Efortul de dezvoltare

Sistemele CRUD au costuri de dezvoltare și operare mai mici. Instrumentele și framework-urile familiare permit dezvoltarea rapidă, depanarea este simplă, iar monitorizarea este relativ simplă.

Implementarea și operarea unui sistem CQRS și Event Sourcing solicită mai mult din partea echipelor de dezvoltare și DevOps. Planificarea atentă a schemelor de evenimente, a brokerilor de mesaje, a garanțiilor de consistență și a infrastructurii de monitorizare este necesară pentru a asigura fiabilitatea.

După o analiză mai detaliată pe criterii, Tabelul 1 prezintă principalele diferențe dintre arhitectura tradițională CRUD și arhitectura CQRS + Event Sourcing.

Tabelul 1

Criteriu	Modelul CRUD	Arhitectura CQRS + Event Sourcing
Complexitatea sistemului	Simple de proiectat, implementat și întreținut.	Complexitate ridicată, necesită expertiză și proiectare atentă.
Scalabilitate	Scalabilitate limitată, dificil de separat operațiile de citire și scriere.	Scalabilitate ridicată, citirile și scrierile pot fi scalate independent.
Consistența datelor	Consistență puternică prin tranzații ACID.	Consistență eventuală, consistența puternică este posibilă, dar mai dificil de obținut.
Performanță	Adecvată pentru sarcini moderate, blocaje la acces concurent intens.	Performanță optimizată prin separarea citirilor de scrieri, mai bună sub sarcină mare.
Audit și analiză istorică	Limitată, necesită mecanisme suplimentare (ex: tabele de audit).	Urmărire completă a istoricului prin evenimente salvate, audit și recuperare integrate.
Efort de dezvoltare și operare	Efort redus de dezvoltare și operare; lansare rapidă pe piață.	Efort ridicat pentru dezvoltare, monitorizare și mentenanță.

Recomandări

Analiza comparativă arată că nu există o arhitectură „mai bună” în mod absolut între CRUD și CQRS combinat cu Event Sourcing. Alegerea optimă depinde în principal de contextul aplicației, scalabilitate, și cerințele specifice ale sistemului.

Arhitectura tradițională CRUD este o alegere potrivită pentru [8]:

- aplicații simple sau cu complexitate medie, unde logica de business nu este complicată.
- echipe mici sau startup-uri care au nevoie de dezvoltare rapidă și time-to-market scurt.
- sisteme care necesită consistență puternică a datelor și au un volum de citiri și scrieri echilibrat.
- proiecte în care dezvoltatorii au experiență limitată cu sisteme distribuite sau arhitecturi complexe.

Exemple de aplicații unde CRUD este potrivit:

- panouri de administrare interne (admin dashboards)
- sisteme de gestiune a utilizatorilor (user management systems)
- aplicații crud standard: gestionarea produselor, comenzilor sau clienților
- sisteme de programări (booking systems)
- platforme de învățare simplificate (lms fără istoric detaliat)
- aplicații de tip to-do list sau notes

Arhitectura CQRS + Event Sourcing se potrivește în special în cazul [9-11]:

- aplicațiilor de mari dimensiuni și cu logică de business complexă, unde citirile și scrierile sunt frecvente și masive.
- sistemelor care necesită scalabilitate separată pentru operațiile de citire și scriere.
- domeniilor în care este nevoie de audit, urmărirea istoricului sau reconstrucția stării (de ex., obligații legale sau analiză istorică).
- aplicațiilor de tip event-driven sau microservicii, unde decuplarea și procesarea asincronă sunt esențiale.
- situațiilor în care este necesară analiza retroactivă a modificărilor de stare sau reconstituirea contextului istoric.

Exemple de aplicații potrivite pentru CQRS + Event Sourcing:

- sisteme financiare sau bancare (ex: evidența tranzacțiilor, istoric de cont)
- aplicații e-commerce mari (ex: amazon, emag) cu volum ridicat de comenzi și produse
- platforme de livrare (ex: glovo, uber eats) unde fiecare comandă și actualizare de stare este un eveniment
- sisteme medicale (emr – electronic medical records) cu istoric detaliat al fiecărui pacient
- platforme de social media (ex: stocarea istorică a postărilor, comentariilor, modificărilor)
- aplicații pentru jocuri multiplayer online (mmorpgs) – unde acțiunile jucătorilor sunt evenimente în timp

Concluzii

Studiul a evidențiat că arhitectura CRUD oferă simplitate, dezvoltare rapidă și este potrivită pentru aplicații cu cerințe moderate, fiind preferată de echipe mici și în fazele inițiale ale proiectelor. Pe de altă parte, arhitectura CQRS combinată cu Event Sourcing excelează în sisteme complexe, distribuite sau cu cerințe stricte de audit, oferind scalabilitate și o mai bună separare a responsabilităților între scriere și citire.

În același timp, s-a constatat că introducerea CQRS și Event Sourcing presupune o pregătire tehnică aprofundată și un efort de implementare și mentenanță mai ridicat.

Recomandarea generală este ca alegerea arhitecturii să fie ghidată de nevoile reale ale aplicației, resursele disponibile și nivelul echipei de dezvoltare. În practică, o soluție hibridă poate reprezenta un compromis echilibrat, în care se utilizează CRUD în modulele simple și CQRS cu Event Sourcing în cele critice sau complexe.

Referințe

- [1] “CRUD Operations Explained – Geek Culture,” *Medium*. Accessed: May 14, 2025. [Online].
Available: <https://medium.com/geekculture/crud-operations-explained-2a44096e9c88>
- [2] “CQRS Design Pattern in Microservices Architectures,” *Design Microservices Architecture with Patterns*. Accessed: May 14, 2025. [Online].
Available: <https://medium.com/design-microservices-architecture-with-patterns/cQRS-design-pattern-in-microservices-architectures-5d41e359768c>
- [3] “Event Sourcing Pattern in Microservices Architectures,” *Design Microservices Architecture with Patterns*. Accessed: May 14, 2025. [Online].
Available: <https://medium.com/design-microservices-architecture-with-patterns/event-sourcing-pattern-in-microservices-architectures-e72bf0fc9274>
- [4] “CQRS vs CRUD: What's the Difference?,” *Spiceworks*. Accessed: May 14, 2025. [Online].
Available: <https://www.spiceworks.com/tech/data-management/articles/cQRS-vs-crud/>
- [5] A. Elsayyad, “CQRS vs CRUD: Choosing the Right Pattern for Your Application,” *Medium*. Accessed: May 14, 2025. [Online].
Available: <https://medium.com/@AmrElsayyad/cQRS-vs-crud-choosing-the-right-pattern-for-your-application-0d614167d12b>
- [6] “Differences Between CQRS and CRUD – System Design,” *GeeksforGeeks*. Accessed: May 14, 2025. [Online].
Available: <https://www.geeksforgeeks.org/differences-between-cQRS-and-crud-system-design/>
- [7] D. Sobrado, “CRUD vs CQRS,” *danielsobrado.com*. Accessed: May 14, 2025. [Online].
Available: <https://www.danielsobrado.com/blog/crud-cQRS/>
- [8] “Differences Between CQRS, MediatR, and CRUD,” *C# Corner*. Accessed: May 14, 2025. [Online].
Available: <https://www.c-sharpcorner.com/article/differences-between-cQRS-mediatr-and-crud/>
- [9] M. Fowler, “CQRS,” *martinfowler.com*. Accessed: May 14, 2025. [Online].
Available: <https://martinfowler.com/bliki/CQRS.html>
- [10] “CQRS Pattern,” *Microsoft Learn – Azure Architecture Center*. Accessed: May 14, 2025. [Online].
Available: <https://learn.microsoft.com/en-us/azure/architecture/patterns/cQRS>
- [11] “Event Sourcing Pattern,” *Microsoft Learn – Azure Architecture Center*. Accessed: May 14, 2025. [Online].
Available: <https://learn.microsoft.com/en-us/azure/architecture/patterns/event-sourcing>